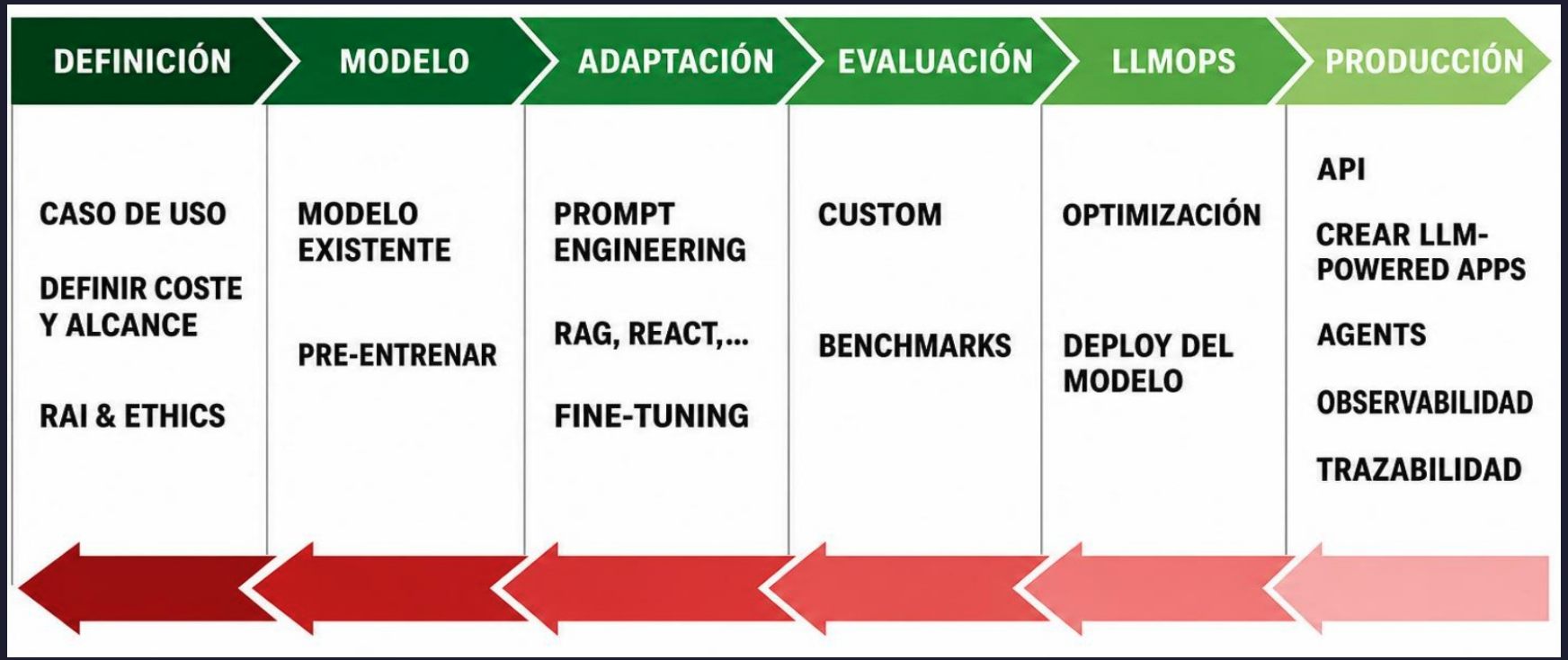
keep coding

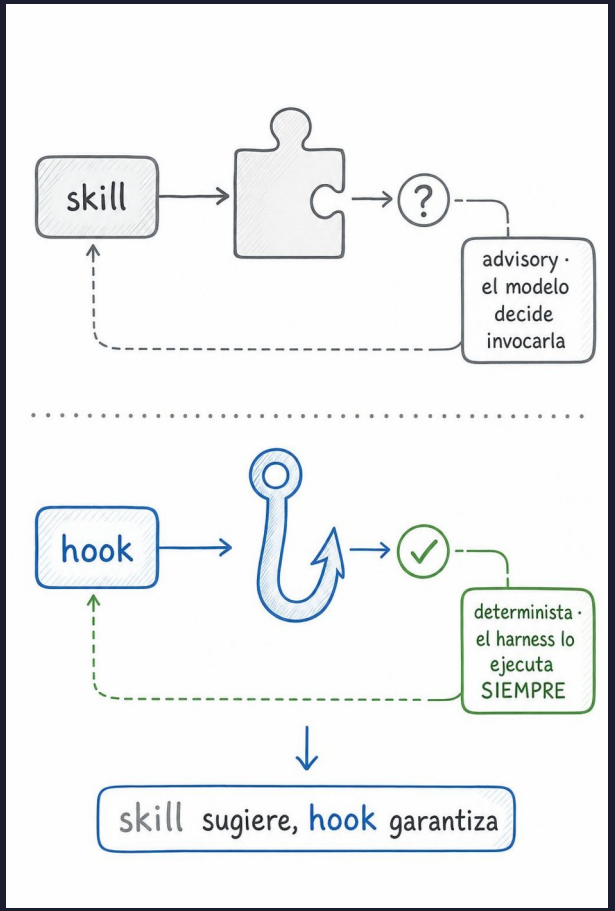
AI ENGINEERING

Hooks


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

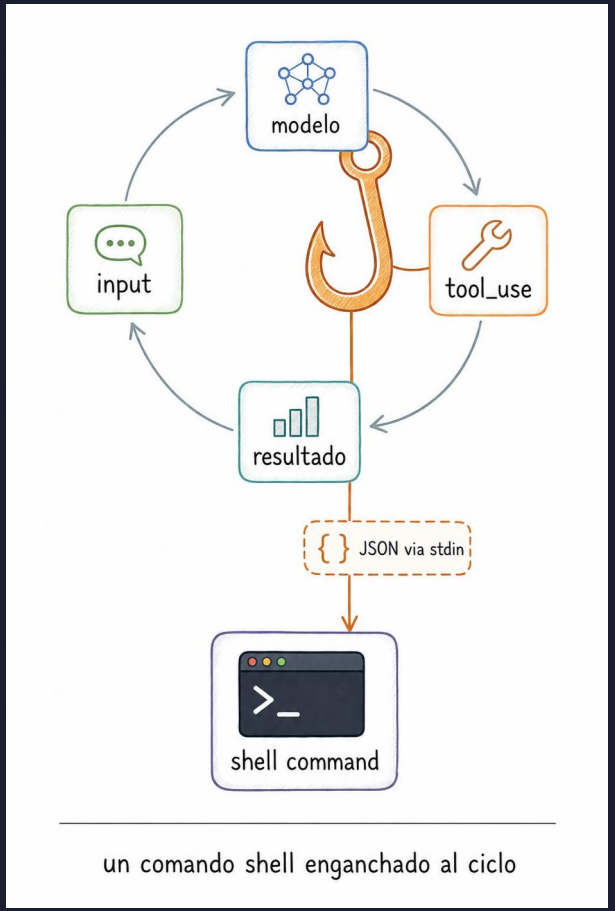
Skill sugiere, hook garantiza

- Una skill SUGIERE: el modelo decide invocarla
- Un hook GARANTIZA: el harness lo ejecuta, no el modelo
- Skill = playbook con criterio · hook = regla con shell
- Si la regla NO puede fallar, es un hook



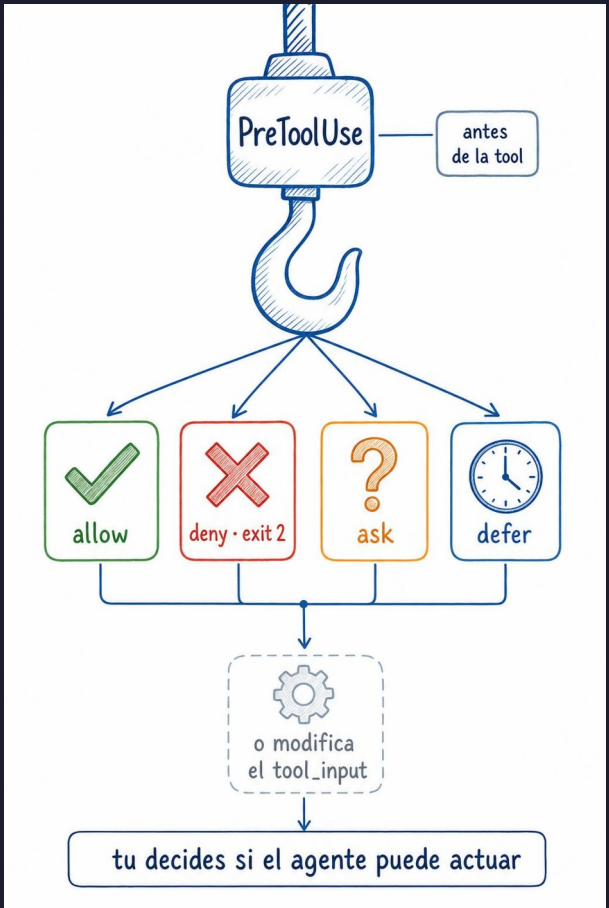
Qué es un hook

- Un comando shell enganchado a un evento del ciclo del agente
- Vive en .claude/settings.json (project) o ~/.claude/settings.json (user)
- Recibe JSON con el contexto (tool, args, etc.) por stdin
- Decide via exit code o JSON · el harness obedece



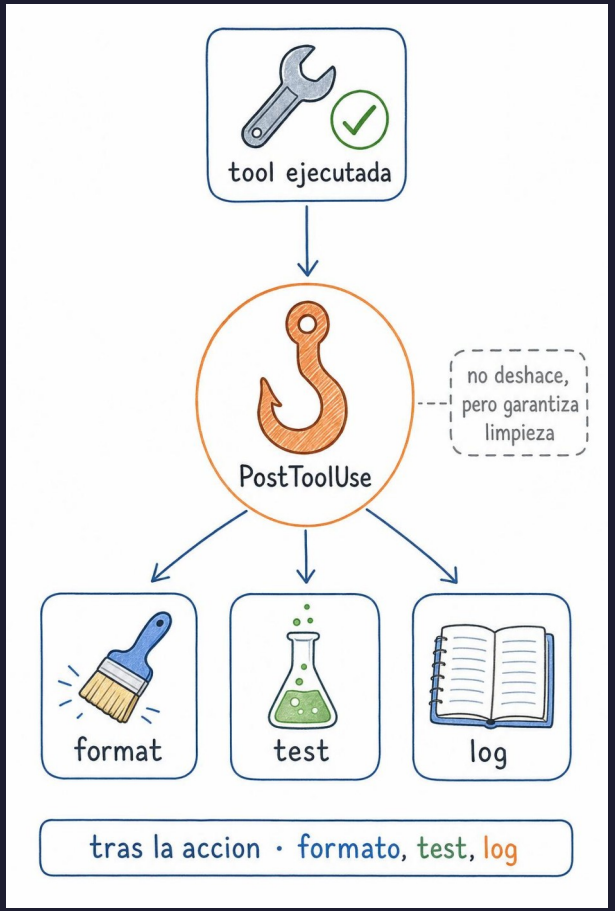
PreToolUse · el mas potente

- Se dispara ANTES de ejecutar la tool
- Cuatro decisiones · allow / deny / ask / defer
- Puede modificar el tool_input antes de ejecutar
- Bloquear · exit code 2 (stderr al modelo) o JSON permissionDecision: deny
- Aquí esta el control de verdad



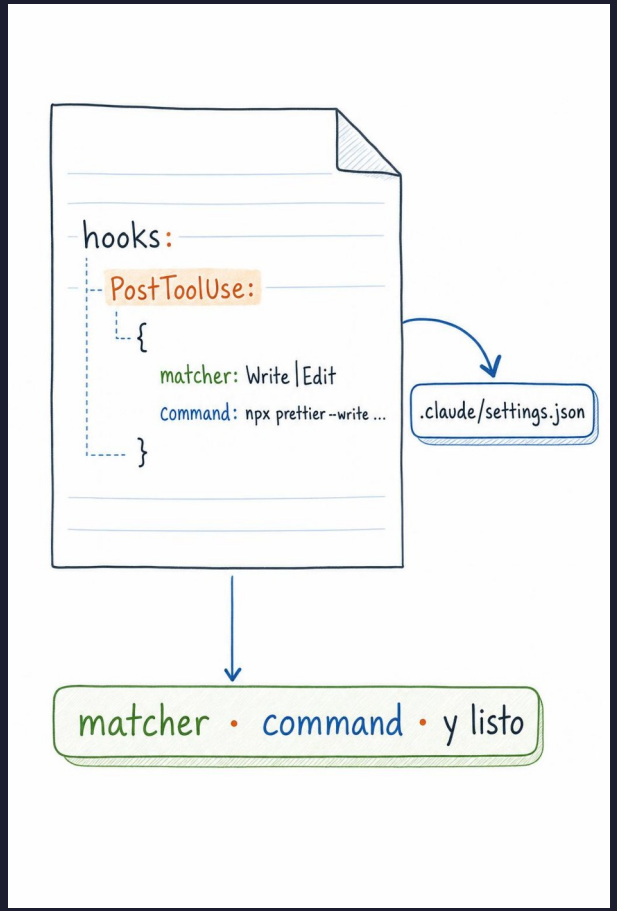
PostToolUse · tras la acción

- Se dispara DESPUES de ejecutar la tool
- No puede deshacer la acción, pero garantiza limpieza
- Casos: auto-format, auto-test, auto-log
- Si falla · el modelo recibe el error y reintenta



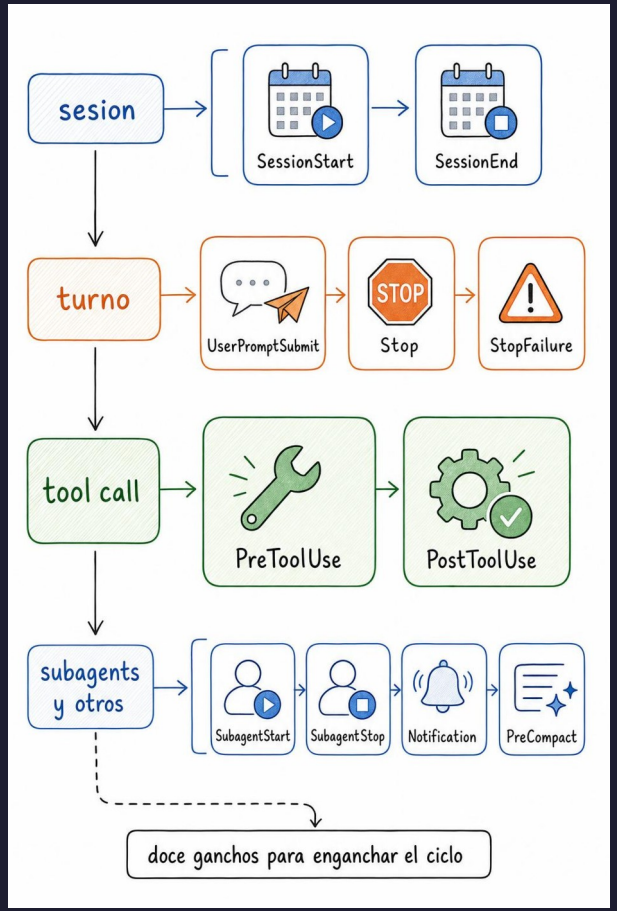
Configuración

- Edita .claude/settings.json o usa el comando /hooks
- Cada hook tiene: evento + matcher (regex de tools) + command
- El matcher 'Write|Edit|MultiEdit' captura las tres tools de edicion
- 4 tipos de handler · command (shell), http, prompt, agent



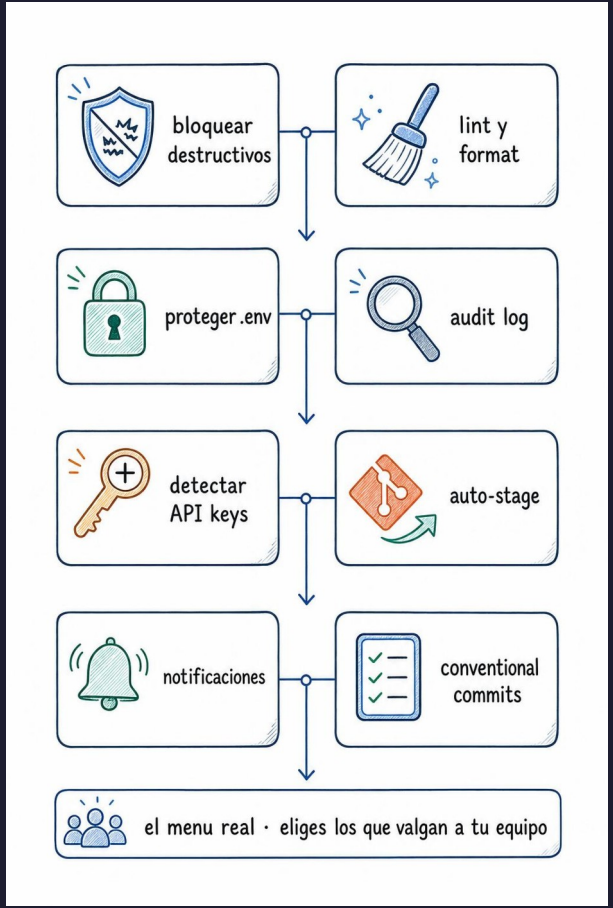
Los 12 eventos · cheat sheet

- Sesión · SessionStart, SessionEnd
- Turno · UserPromptSubmit, Stop, StopFailure
- Tool call · PreToolUse, PostToolUse (los mas usados)
- Subagents · SubagentStart, SubagentStop
- Otros · Notification, PreCompact, PostCompact
- Doce puntos donde meter mano al ciclo



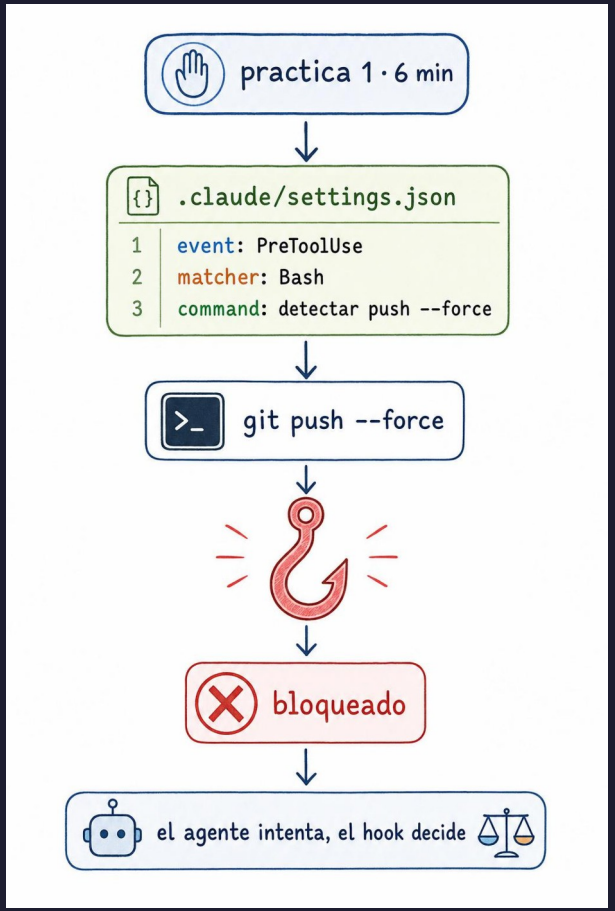
Patrones en producción

- Bloquear destructivos (rm -rf, force push, drop table)
- Lint y format automático en cada Edit
- Proteger secretos (.env, credentials, *.key)
- Audit log de cada tool usada
- Secret scanning · detectar API keys en código nuevo
- Notificaciones desktop / Slack al terminar
- Conventional commits enforcement
- SessionStart inyecta git status (contexto fresco)



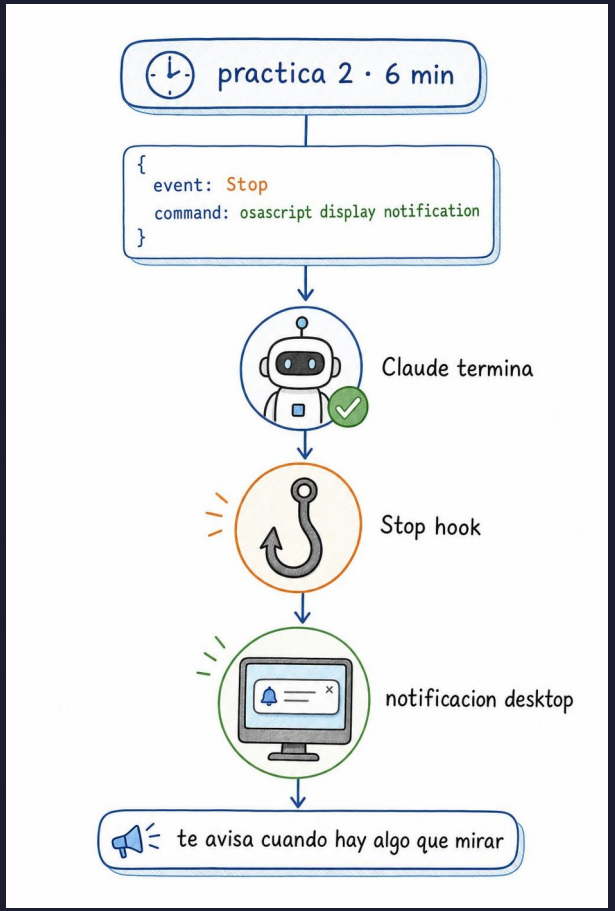
Práctica 1 · force push

- Evento: PreToolUse · Matcher: Bash
- Lee stdin JSON, mira el campo tool_input.command
- Si contiene 'push --force' o 'push -f' -> exit 2 con stderr
- Lo metemos en .claude/settings.json del proyecto
- Probamos: el agente intenta, el hook bloquea



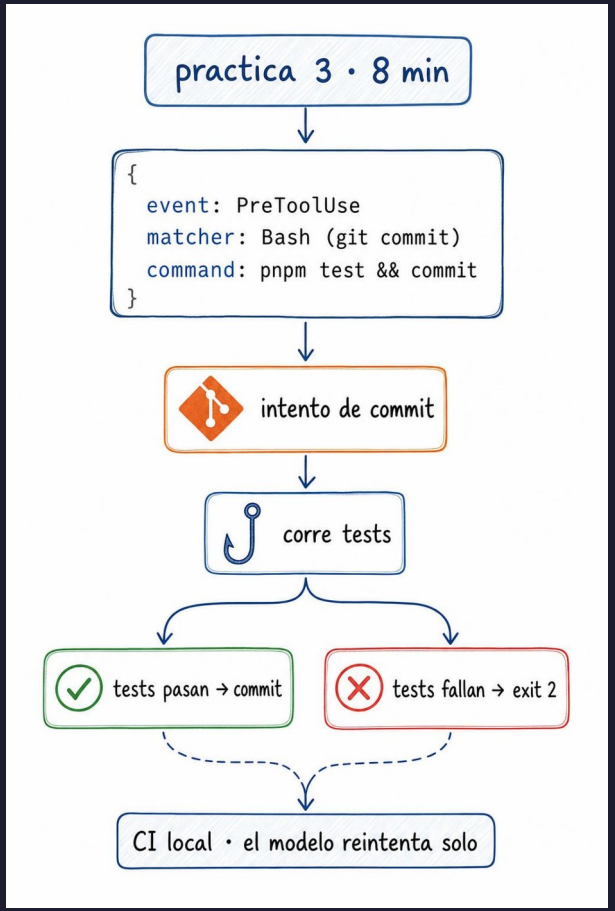
Práctica 2 · notificacion

- Evento: Stop · sin matcher (se dispara cuando Claude pide tu input)
- Command: osascript -e 'display notification "Claude listo"'
- También vale terminal-notifier, o curl a Slack webhook
- Te vas a hacer cafe y vuelves cuando te llama
- Cambia tu workflow · no vigilas el terminal



Práctica 3 · tests pre-commit

- Evento: PreToolUse · Matcher: Bash filtrando 'git commit'
- Command: corre pnpm test antes
- Si fallan: exit 2 + stderr -> Claude lo ve y arregla
- Si pasan: el commit procede normal
- CI-like local · el modelo reintenta solo




keep coding